

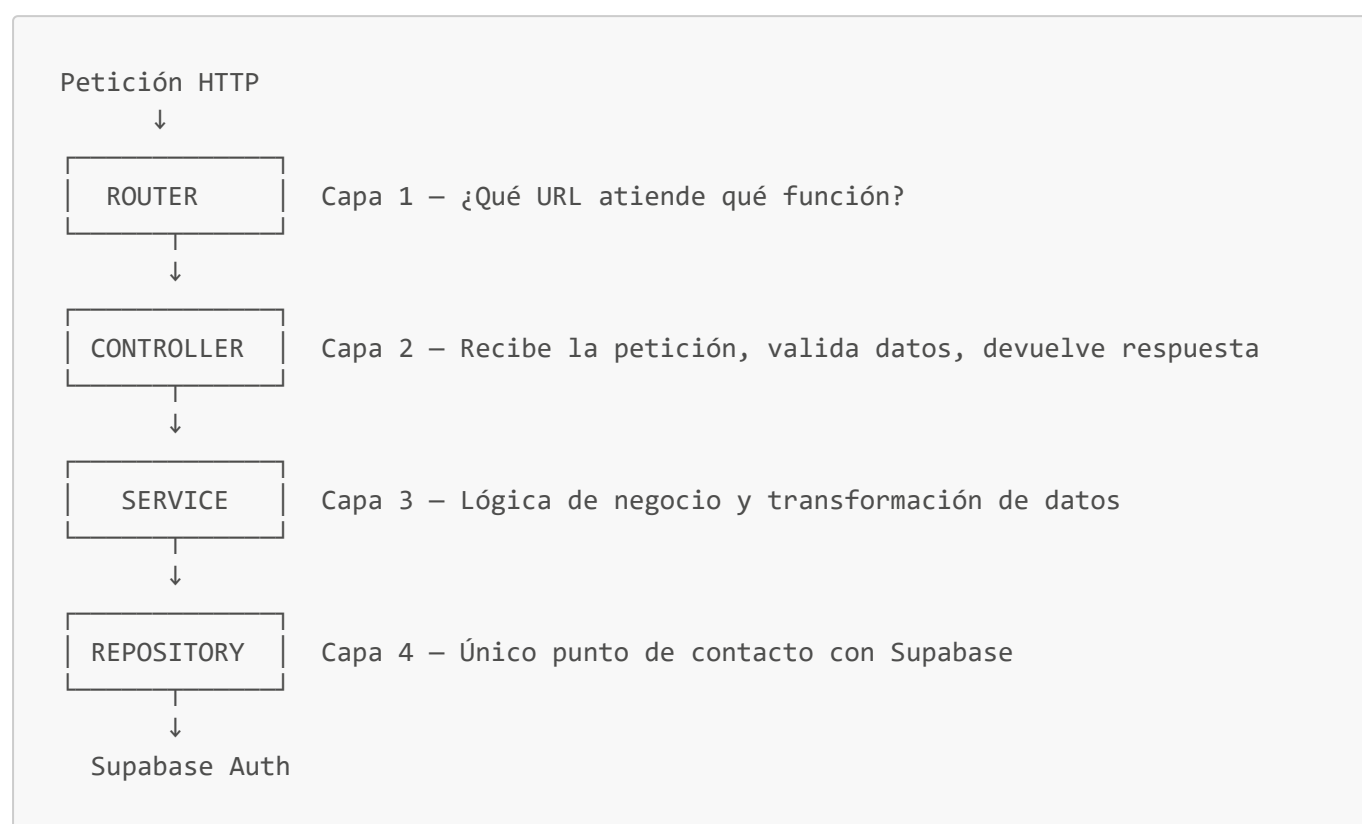
Guía Práctica — M1: Login y Autenticación con Supabase

¿Qué construimos?

Una API REST con Node.js + Express que permite a los usuarios de un portal bancario iniciar sesión, cerrar sesión y consultar su perfil. La autenticación la maneja **Supabase Auth** (servicio externo), y nuestra API actúa como intermediaria.

Arquitectura en capas

El proyecto usa una arquitectura de **4 capas**. Cada capa tiene una única responsabilidad y solo se comunica con la capa inmediatamente inferior.



¿Por qué este orden? Separar responsabilidades hace que el código sea más fácil de mantener, probar y modificar. Si mañana cambiamos Supabase por otro proveedor de autenticación, solo tocamos el Repository.

Orden de escritura del código y por qué

El código se escribió de **adentro hacia afuera**, desde la base de datos hasta la entrada HTTP. Esto es una buena práctica porque cada capa depende de la que está debajo, no al revés.

Paso 1 — `.env` (Variables de entorno)

```
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_KEY=tu_anon_key
PORT=3000
```

Por qué primero: Sin las credenciales de Supabase no puede funcionar nada. Se escriben antes que cualquier código para tenerlas disponibles desde el inicio. Nunca se suben al repositorio (agregar `.env` al `.gitignore`).

Paso 2 — `src/config/supabase.js` (Cliente Supabase)

```
const { createClient } = require('@supabase/supabase-js');

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);

module.exports = supabase;
```

Por qué segundo: Es la conexión con Supabase. Se crea una sola instancia (`singleton`) que se comparte en toda la aplicación. Si se creara una instancia en cada archivo, se abriría una nueva conexión con cada petición, lo que consume más recursos innecesariamente.

Paso 3 — `src/repositories/authRepository.js` (Capa 4)

```
const supabase = require('../config/supabase');

exports.signIn = async (email, password) => {
  const { data, error } = await supabase.auth.signInWithPassword({ email, password });
  if (error) throw new Error('Credenciales incorrectas. Inténtalo nuevamente.');
```

```
  return data;
};

exports.signOut = async () => {
  const { error } = await supabase.auth.signOut();
  if (error) throw new Error('Error al cerrar sesión.');
```

```
};

exports.getUser = async (token) => {
  const { data, error } = await supabase.auth.getUser(token);
  if (error) throw new Error('Token inválido o expirado.');
```

```
  return data;
};
```

Por qué tercero: Es la capa más cercana a los datos. Se escribe antes que las capas superiores porque ellas dependen de lo que este archivo exporta. Contiene exactamente 3 funciones, una por cada operación de Supabase Auth.

Detalle importante: Supabase devuelve `{ data, error }`. Si hay error, lo convertimos en una excepción con un mensaje amigable para el usuario.

Paso 4 — `src/services/authService.js` (Capa 3)

```
const authRepository = require('../repositories/authRepository');

exports.login = async (email, password) => {
  const data = await authRepository.signIn(email, password);
  return {
    usuario: {
      id: data.user.id,
      email: data.user.email,
      nombre: data.user.user_metadata?.full_name || 'Cliente'
    },
    token: data.session.access_token
  };
};

exports.logout = async () => {
  await authRepository.signOut();
};

exports.getUsuarioActual = async (token) => {
  const data = await authRepository.getUser(token);
  return {
    id: data.user.id,
    email: data.user.email,
    nombre: data.user.user_metadata?.full_name || 'Cliente'
  };
};
```

Por qué cuarto: Aquí está la **lógica de negocio**. El Service no sabe nada de Supabase; solo llama al Repository y transforma la respuesta.

Transformación que hace: Supabase devuelve un objeto grande con muchos campos internos. El Service extrae solo lo que necesita el cliente: `id`, `email`, `nombre` y `token`. Así el Controller nunca recibe datos en crudo de Supabase.

El operador `?.` en `data.user.user_metadata?.full_name` significa: "si `user_metadata` existe, dame `full_name`; si no existe, no falles, devuelve `undefined`". El `|| 'Cliente'` pone un valor por defecto.

Paso 5 — `src/controllers/authController.js` (Capa 2)

```
const authService = require('../services/authService');

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ success: false, message: 'Email y password son obligatorios' });
    }

    const resultado = await authService.login(email, password);
    res.json({ success: true, data: resultado });

  } catch (error) {
    res.status(401).json({ success: false, message: error.message });
  }
};
```

Por qué quinto: El Controller es el "portero". Su trabajo es:

1. Leer los datos de la petición (`req.body`, `req.headers`)
2. Validar que los datos obligatorios estén presentes
3. Llamar al Service
4. Devolver la respuesta JSON con el formato correcto

No contiene lógica de negocio. Si el Service lanza un error, el `catch` lo atrapa y devuelve una respuesta de error al cliente.

Paso 6 — `src/routes/authRoutes.js` (Capa 1)

```
const express = require('express');
const router = express.Router();
const controller = require('../controllers/authController');

router.post('/login', controller.login);
router.post('/logout', controller.logout);
router.get('/me', controller.getMe);

module.exports = router;
```

Por qué sexto: El Router es solo un mapa de URLs. Se escribe al final porque para conectar una URL a un controller, el controller ya debe existir.

Paso 7 — `app.js` (Servidor principal)

```

const express = require('express');
const cors    = require('cors');
require('dotenv').config();

const authRoutes = require('./src/routes/authRoutes');

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', authRoutes);

app.get('/', (req, res) => {
  res.json({ mensaje: 'API Portal Mi Banco funcionando correctamente' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Servidor corriendo en puerto ${PORT}`));

```

Por qué último: Es el punto de entrada de la aplicación. Registra todos los middlewares y rutas, y arranca el servidor. Se escribe al final porque necesita que todas las rutas ya estén definidas.

`require('dotenv').config()` debe llamarse antes que cualquier otra cosa para que las variables del `.env` estén disponibles cuando el resto del código las necesite.

Estructura de archivos

```

s2_m1_supabase/
├── .env                ← Variables de entorno (privado)
├── app.js              ← Punto de entrada del servidor
├── package.json
├── src/
│   ├── config/
│   │   └── supabase.js    ← Cliente Supabase (singleton)
│   ├── repositories/
│   │   └── authRepository.js ← Capa 4: acceso a Supabase Auth
│   ├── services/
│   │   └── authService.js  ← Capa 3: lógica de negocio
│   ├── controllers/
│   │   └── authController.js ← Capa 2: manejo de peticiones HTTP
│   └── routes/
│       └── authRoutes.js   ← Capa 1: definición de URLs

```

Endpoints del M1

01 — GET /

Verifica que el servidor esté corriendo.

Campo	Valor
Método	GET
URL	http://localhost:3000/
Body	Ninguno
Auth	No requerida

Respuesta exitosa (200):

```
{
  "mensaje": "API Portal Mi Banco funcionando correctamente"
}
```

02 — POST /api/auth/login

Inicia sesión con email y contraseña. Devuelve los datos del usuario y un token JWT.

Campo	Valor
Método	POST
URL	http://localhost:3000/api/auth/login
Body	JSON con email y password
Auth	No requerida

Body de la petición:

```
{
  "email": "cliente01@bbva.com",
  "password": "12345678"
}
```

Respuesta exitosa (200):

```
{
  "success": true,
  "data": {
    "usuario": {
      "id": "f03363d1-c24d-463b-a654-4c38dd590007",
      "email": "cliente01@bbva.com",
      "nombre": "Cliente"
    },
  },
}
```

```
{
  "token": "eyJhbGciOiJIUzI1NiIsI..."
}
```

Respuesta de error (401):

```
{
  "success": false,
  "message": "Credenciales incorrectas. Inténtalo nuevamente."
}
```

Importante: Guarda el `token` de la respuesta para usarlo en el endpoint `/me`.

03 — POST `/api/auth/logout`

Cierra la sesión activa en Supabase.

Campo	Valor
Método	POST
URL	<code>http://localhost:3000/api/auth/logout</code>
Body	Ninguno
Auth	No requerida

Respuesta exitosa (200):

```
{
  "success": true,
  "message": "Sesión cerrada correctamente"
}
```

04 — GET `/api/auth/me`

Devuelve los datos del usuario autenticado. Requiere enviar el token en el header.

Campo	Valor
Método	GET
URL	<code>http://localhost:3000/api/auth/me</code>
Body	Ninguno
Auth	Header <code>Authorization: Bearer <token></code>

Header requerido:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsI...
```

Respuesta exitosa (200):

```
{
  "success": true,
  "data": {
    "id": "f03363d1-c24d-463b-a654-4c38dd590007",
    "email": "cliente01@bbva.com",
    "nombre": "Cliente"
  }
}
```

Respuesta de error sin token (401):

```
{
  "success": false,
  "message": "Token requerido"
}
```

Flujo completo de una petición de login

1. Postman envía POST /api/auth/login con { email, password }
- ↓
2. Express recibe la petición y la pasa al Router
- ↓
3. Router identifica la ruta y llama a controller.login
- ↓
4. Controller valida que email y password existan en el body
- ↓
5. Controller llama a authService.login(email, password)
- ↓
6. Service llama a authRepository.signIn(email, password)
- ↓
7. Repository llama a supabase.auth.signInWithPassword({ email, password })
- ↓
8. Supabase verifica las credenciales y devuelve { data, error }
- ↓
9. Repository devuelve data al Service (o lanza error si falló)
- ↓
10. Service extrae id, email, nombre y token del objeto de Supabase
- ↓
11. Controller recibe el resultado y responde con { success: true, data: ... }



12. Postman muestra la respuesta JSON con el token

Conceptos clave

Concepto	Explicación
JWT (token)	Cadena cifrada que identifica al usuario. Se genera al hacer login y se envía en cada petición privada.
Bearer	Prefijo que indica que el header Authorization contiene un token JWT.
async/await	Permite escribir código asíncrono (que espera respuestas) de forma legible.
try/catch	Captura errores para devolver una respuesta controlada en lugar de romper el servidor.
singleton	Patrón que garantiza que solo existe una instancia de un objeto (el cliente Supabase).
dotenv	Librería que carga las variables del archivo <code>.env</code> en <code>process.env</code> .